

Attention Mechanisms

Chapter 6 · Learning where to look

Dr. A. Belcaid

Table of contents

1 The encoder bottleneck **4**
1.1 Recall: one RNN reads, another writes . . . 4
1.2 The bottleneck is storage and access . . . 5
1.3 Section synthesis . . . 6

2 Attention as soft lookup **7**
2.1 Keep every source memory . . . 7
2.2 Three operations: score, normalize, retrieve . . . 8
2.3 Worked soft lookup . . . 9
2.4 What learning changes . . . 10
2.5 Section synthesis . . . 10

3 Cross-attention **10**
3.1 Queries, keys, and values are roles . . . 10
3.2 Scoring functions . . . 11
3.2.1 Dot product . . . 11
3.2.2 Additive attention . . . 11
3.3 One decoder query creates one alignment . . . 11
3.4 A new query creates a new alignment . . . 12
3.5 Section synthesis . . . 13

4 Self-attention **13**
4.1 Change the origin, keep the mechanism . . . 13
4.2 A pronoun performs a lookup . . . 14
4.3 Every token asks its own question . . . 14
4.4 Shorter paths and parallel computation . . . 15
4.5 Section synthesis . . . 15

5 Modern attention **16**
5.1 Why scale dot products? . . . 16

5.2 Worked scaling example . . . 17
5.3 Causal masking: do not read tomorrow . . . 17
5.4 Why use multiple heads? . . . 18
5.5 Reading attention visualizations . . . 19
5.5.1 Attention heatmap . . . 19
5.5.2 Connection view . . . 19
5.6 Modern attention in one view . . . 20

6 The Transformer bridge **20**
6.1 Self-attention has no order by itself . . . 20
6.2 Attention mixes positions, not features deeply . . . 21
6.3 Deep stacks need safe routes . . . 21
6.4 Final visual bridge . . . 22

7 Chapter summary **22**

8 Further reading and exploration **23**
8.1 Defining papers . . . 23
8.2 Visual explanations . . . 23
8.3 Attention patterns and interpretation . . . 23
8.4 References . . . 23

The recurrent encoder–decoder gave us a general recipe for conditional sequence generation: read an input sequence, compress it into a state, and generate an output sequence. Its central weakness was equally simple: **every source detail had to cross one fixed-size vector**. Attention repairs this bottleneck by allowing a prediction to retrieve information directly from all available source states.

This chapter develops attention as an inspectable soft lookup. We will keep one small translation example visible while deriving scores, weights, and context vectors; then we will reuse the same mechanism inside a single sequence. The chapter ends at the boundary of the Transformer architecture. We will identify what a complete model still needs, but leave the construction of the Transformer block to the next chapter.

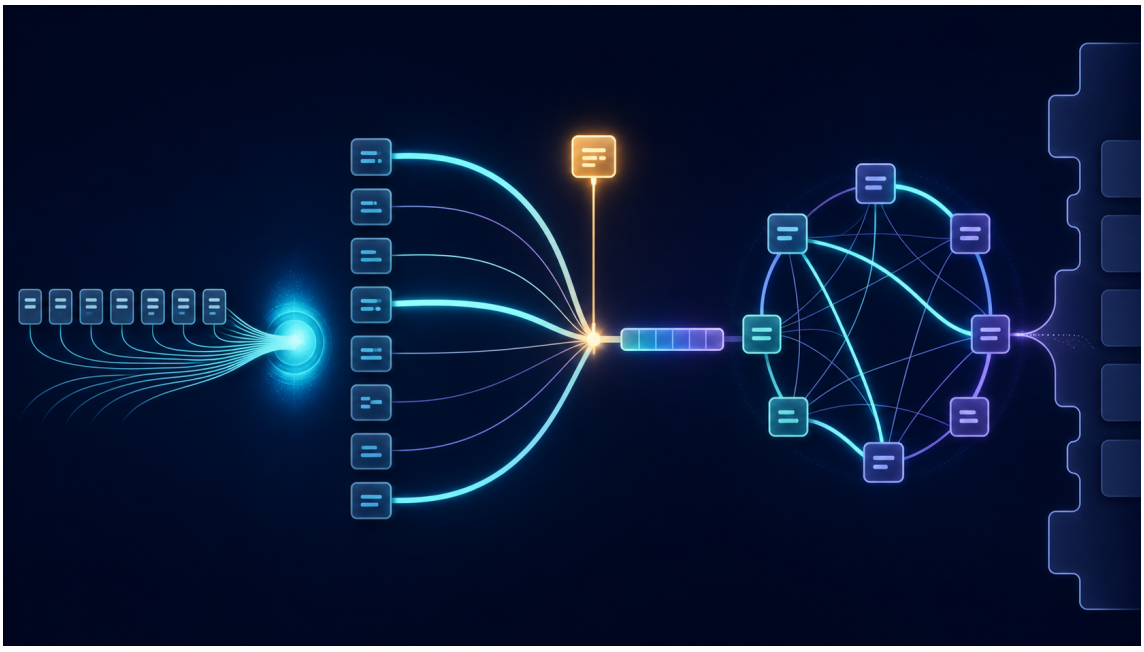


Figure 1: The chapter journey from a recurrent bottleneck through soft lookup, cross-attention, self-attention, scaled and masked multi-head attention, and an unfinished bridge toward the Transformer block.

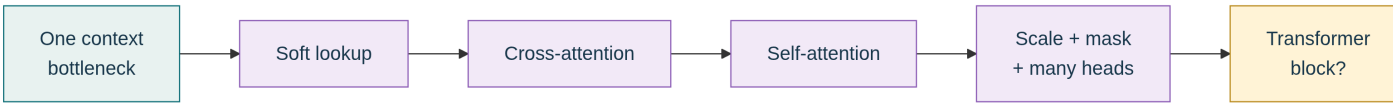


Figure 2: Each part keeps the same score–normalize–retrieve operation while changing where queries, keys, and values come from.

Learning objectives

After this chapter, you should be able to:

1. explain why one encoder state creates both a storage and an access bottleneck;
2. calculate attention scores, softmax weights, and a weighted context vector;
3. distinguish queries, keys, and values by their computational roles;
4. distinguish cross-attention from self-attention;
5. derive the matrix form of scaled dot-product attention;
6. construct and interpret a causal mask;
7. explain what multiple heads add beyond one attention distribution;
8. interpret attention visualizations cautiously; and
9. identify the missing ingredients that motivate a Transformer block.

1 The encoder bottleneck

1.1 Recall: one RNN reads, another writes

Let the source sentence be:

```
Le colis arrivera demain
```

and the desired translation be:

```
The package will arrive tomorrow
```

An encoder RNN reads the source one token at a time:

$$h_i = f_{\text{enc}}(h_{i-1}, x_i), \qquad i = 1, \dots, n. \tag{1}$$

In the simplest recurrent encoder–decoder, only the final state is passed to the decoder:

$$c = h_n, \qquad s_0 = c. \tag{2}$$

The decoder then produces one target state and token distribution at a time:

$$s_t = f_{\text{dec}}(s_{t-1}, y_{t-1}), \qquad p(y_t \mid y_{<t}, x) = \text{softmax}(W_o s_t + b_o). \tag{3}$$

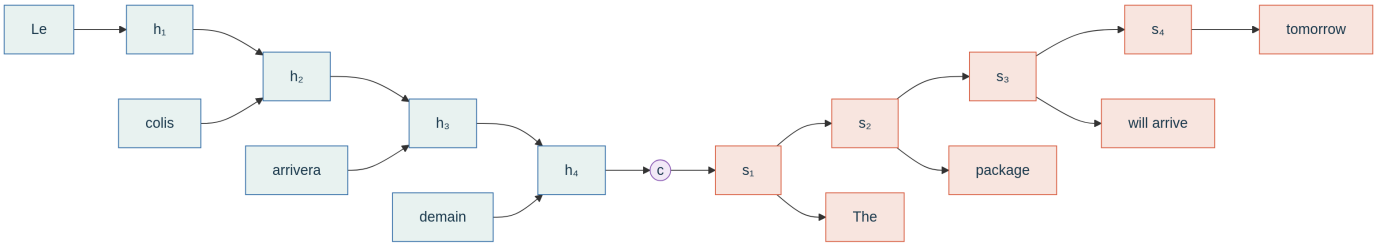


Figure 3: The entire source sequence reaches every decoder step through the single context vector $c = h_4$.

This architecture was historically important: sequence-to-sequence learning showed that a neural model could learn variable-length mappings end to end (Sutskever et al. 2014). The fixed vector, however, forces every source detail—content, order, syntax, entities, and ambiguities—through the same narrow channel.

1.2 The bottleneck is storage and access

The problem is often described as insufficient storage: one vector must remember an entire sentence. That is true, but incomplete. Even if a large vector could store every detail, the decoder would still receive the **same undifferentiated vector at every output step**.

Table 1: Different target predictions need different parts of the source.

Decoder step	Desired output	Most relevant source evidence
1	*The*	source noun phrase and target syntax
2	*package*	*colis*
3	*will arrive*	*arrivera*
4	*tomorrow*	*demain*

The decoder should not have to unpack the same vector in the same way for every token. It needs an **access mechanism**: at step t, form a request that selects the source information useful for that prediction.

> **!** **The change in one sentence**
>
> Do not discard $h_1, \dots, h_n$. Keep every encoder state available, and let each decoder step retrieve a different weighted combination.

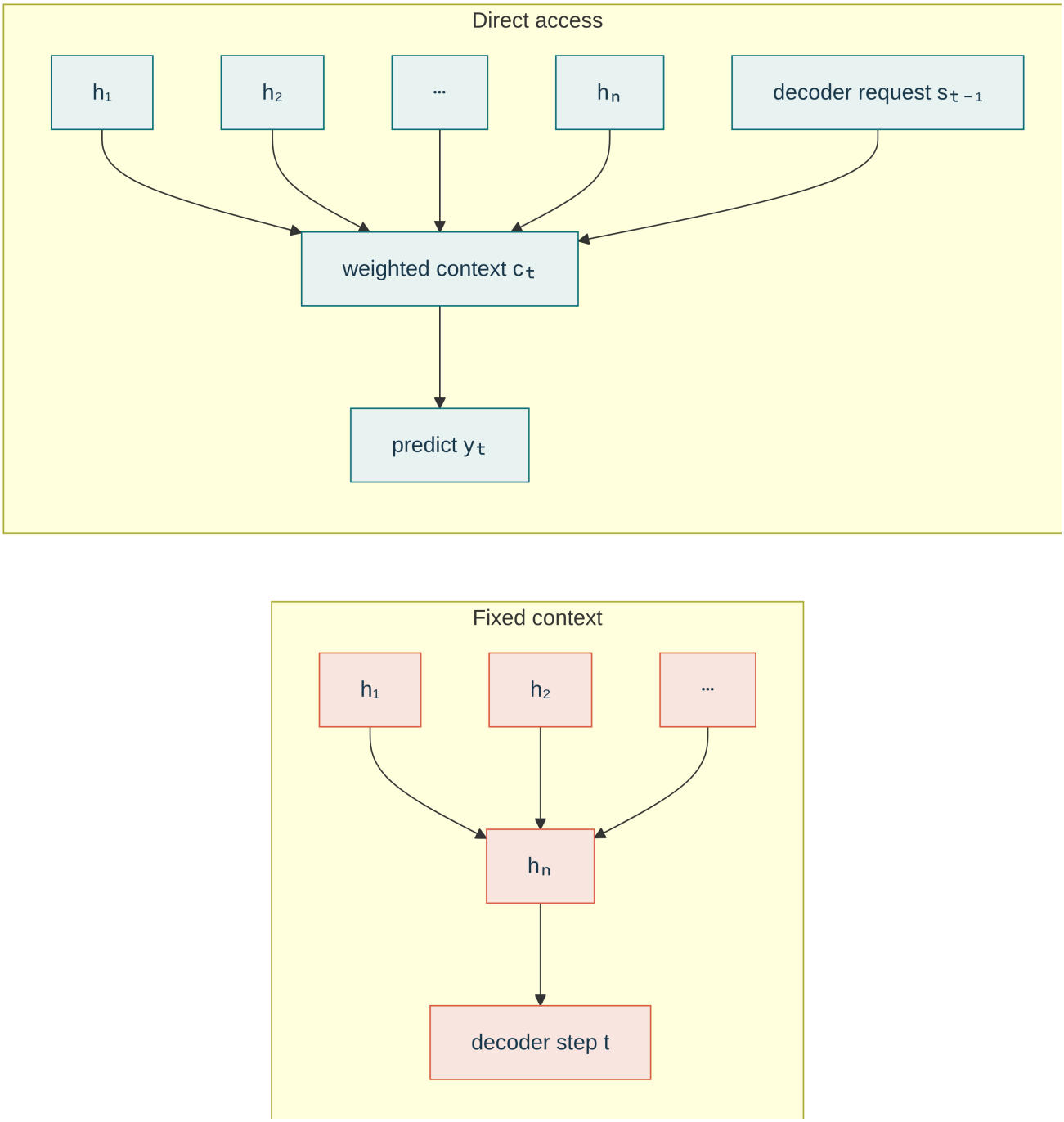


Figure 4: Attention replaces one mandatory path through h_n with direct, prediction-dependent access to all encoder memories.

Bahdanau, Cho, and Bengio introduced this learned alignment mechanism for neural machine translation (Bahdanau et al. 2015). Luong, Pham, and Manning later systematized several global and local variants (Luong et al. 2015).

1.3 Section synthesis

The recurrent encoder–decoder compresses because it discards intermediate states. Attention begins with a simple reversal of that decision: preserve the states and learn how

to access them.

The next question is computational: **how can a model select one memory while remaining differentiable end to end?**

2 Attention as soft lookup

2.1 Keep every source memory

After encoding four tokens, retain the matrix of source states:

$$H = \begin{bmatrix} h_1^\top \\ h_2^\top \\ h_3^\top \\ h_4^\top \end{bmatrix} \in \mathbb{R}^{4 \times d_h}. \tag{4}$$

At decoder step t, the previous decoder state s_{t-1} summarizes what has already been generated. It therefore acts as a request. Informally:

> "Given what I have translated so far, which source memory should influence the next word?"

A hard lookup would choose one index i. That discrete choice is awkward to train with ordinary backpropagation. Attention instead performs a **soft lookup**: assign every memory a nonnegative weight, make the weights sum to one, and retrieve their weighted average.

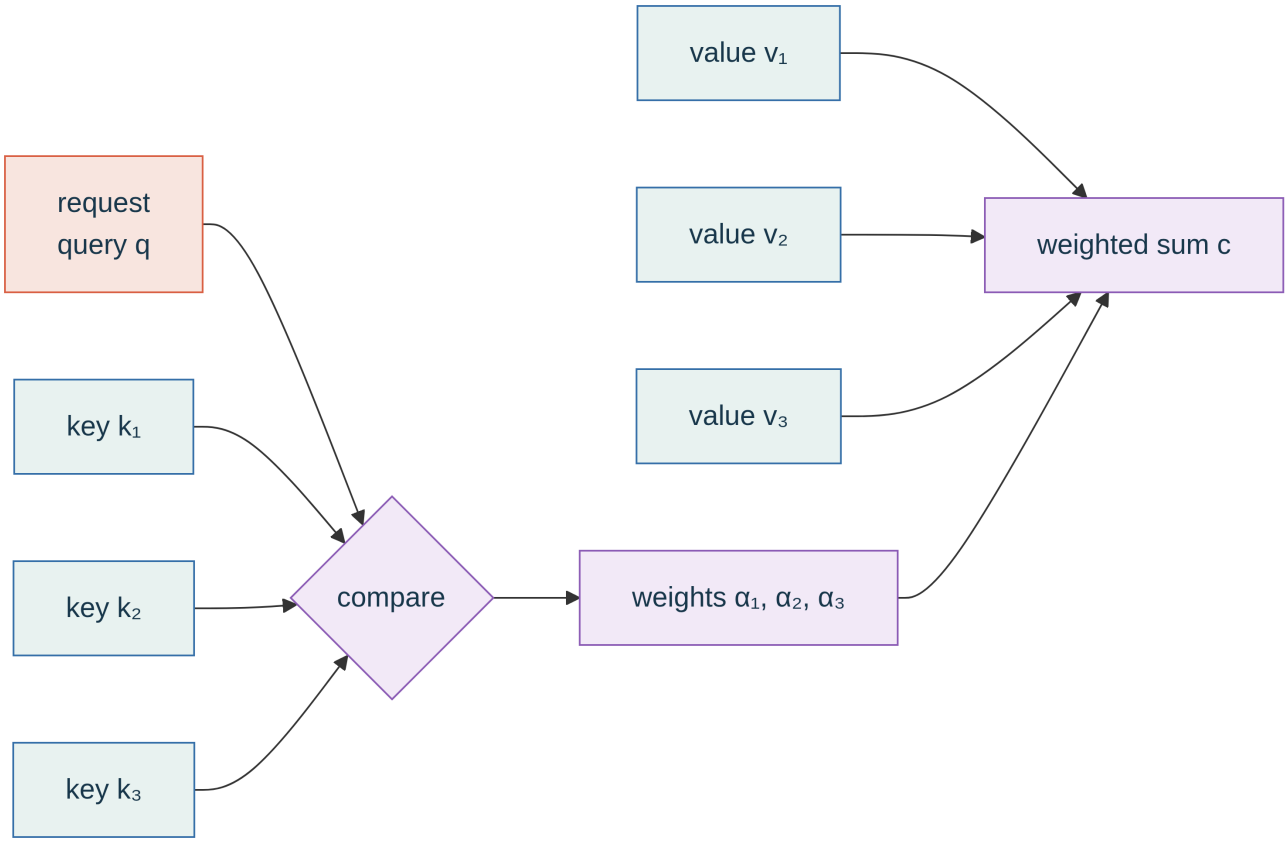


Figure 5: A query compares with every key; normalized weights determine how much of every value is retrieved.

2.2 Three operations: score, normalize, retrieve

For a query q and memory pairs (k_i, v_i):

1. **Score** each query–key pair:

$$e_i = \text{score}(q, k_i). \tag{5}$$

2. **Normalize** all scores with softmax:

$$\alpha_i = \frac{\exp(e_i)}{\sum_j \exp(e_j)}. \tag{6}$$

3. **Retrieve** a weighted combination of values:

$$c = \sum_i \alpha_i v_i. \tag{7}$$

The weights satisfy $\alpha_i \geq 0$ and $\sum_i \alpha_i = 1$. The context is therefore a convex combination of values.

Pause · exact calculation

Suppose one query produces the scores

$$e = [1, 3, 0, 2]$$

for four scalar values

$$v = [2, 5, -1, 3].$$

1. Calculate $\alpha = \text{softmax}(e)$.
2. Calculate $c = \sum_i \alpha_i v_i$.
3. Which value contributes most, and why?

2.3 Worked soft lookup

First exponentiate the scores:

Table 2: Every intermediate quantity in the soft lookup. Values are rounded to three decimals.

Memory i	Score e_i	$\exp(e_i)$	Weight α_i	Value v_i	Contribution $\alpha_i v_i$
1	1	2.718	0.087	2	0.174
2	3	20.086	0.644	5	3.220
3	0	1.000	0.032	−1	−0.032
4	2	7.389	0.237	3	0.711
Total		**31.193**	**1.000**		**4.073**

Thus,

$$\alpha \approx [0.087, 0.644, 0.032, 0.237]$$

and

$$c \approx 0.087(2) + 0.644(5) + 0.032(-1) + 0.237(3) = 4.073.$$

Memory 2 contributes most because it has the largest compatibility score. Notice that the other memories are not discarded: a soft lookup can combine evidence.

Numerical stability

Implement softmax after subtracting the maximum score:

$$\text{softmax}(e)_i = \frac{\exp(e_i - \max_j e_j)}{\sum_k \exp(e_k - \max_j e_j)}.$$

This leaves the probabilities unchanged while preventing unnecessarily large exponentials.

2.4 What learning changes

No human assigns the scores in Table 2. The scoring parameters are trained through the final task loss. If attending to *demain* helps predict *tomorrow*, backpropagation adjusts the projections so that similar query–key pairs receive larger scores in future examples.

The attention weights are intermediate differentiable variables:

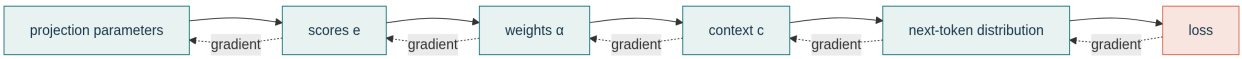


Figure 6: The translation loss supervises the attention mechanism through the context vector.

2.5 Section synthesis

Attention is not a mysterious new memory type. It is a differentiable retrieval rule:

$$\boxed{\text{score} \longrightarrow \text{normalize} \longrightarrow \text{retrieve}}.$$

We have not yet specified what the query, keys, and values represent. Cross-attention gives those roles a concrete meaning in translation.

3 Cross-attention

3.1 Queries, keys, and values are roles

In encoder–decoder attention:

Table 3: The same source state can be projected once as a key and once as a value.

Role	Comes from	Question answered
Query q_t	current or previous decoder state	What information is needed now?
Keys k_i	encoder states	Which source position matches the request?
Values v_i	encoder states	What content should be retrieved?

A common projected form is

$$q_t = s_{t-1}W_Q, \qquad k_i = h_i W_K, \qquad v_i = h_i W_V. \tag{8}$$

The key and value originate from the same source state but serve different purposes. A useful analogy is a library:

- the **key** is the catalog description used to decide relevance;
- the **value** is the book content retrieved after the match.

3.2 Scoring functions

Several differentiable compatibility functions are possible.

3.2.1 Dot product

$$e_{ti} = q_t^\top k_i. \tag{9}$$

Dot products are efficient and become the standard form in the Transformer.

3.2.2 Additive attention

$$e_{ti} = v_a^\top \tanh(W_q q_t + W_k k_i + b_a). \tag{10}$$

This small neural network was used in the original alignment model (Bahdanau et al. 2015). Additive and multiplicative variants can learn similar behaviors, but their dimensions, parameter counts, and computational profiles differ (Luong et al. 2015).

3.3 One decoder query creates one alignment

Suppose the decoder is ready to produce *tomorrow*. Its query is compared with all French source keys:

Table 4: A plausible alignment for generating *tomorrow*.

Source position i	1	2	3	4
Token	Le	colis	arrivera	demain
Score e_{ti}	0.2	0.1	0.8	2.6
Weight α_{ti}	0.069	0.062	0.126	**0.743**

The context for this decoder step is

$$c_t = \sum_{i=1}^{4} \alpha_{ti} v_i. \tag{11}$$

The next-token distribution can use both decoder state and retrieved context:

$$p(y_t \mid y_{<t}, x) = \text{softmax}(W_o[s_t; c_t] + b_o)\,. \tag{12}$$

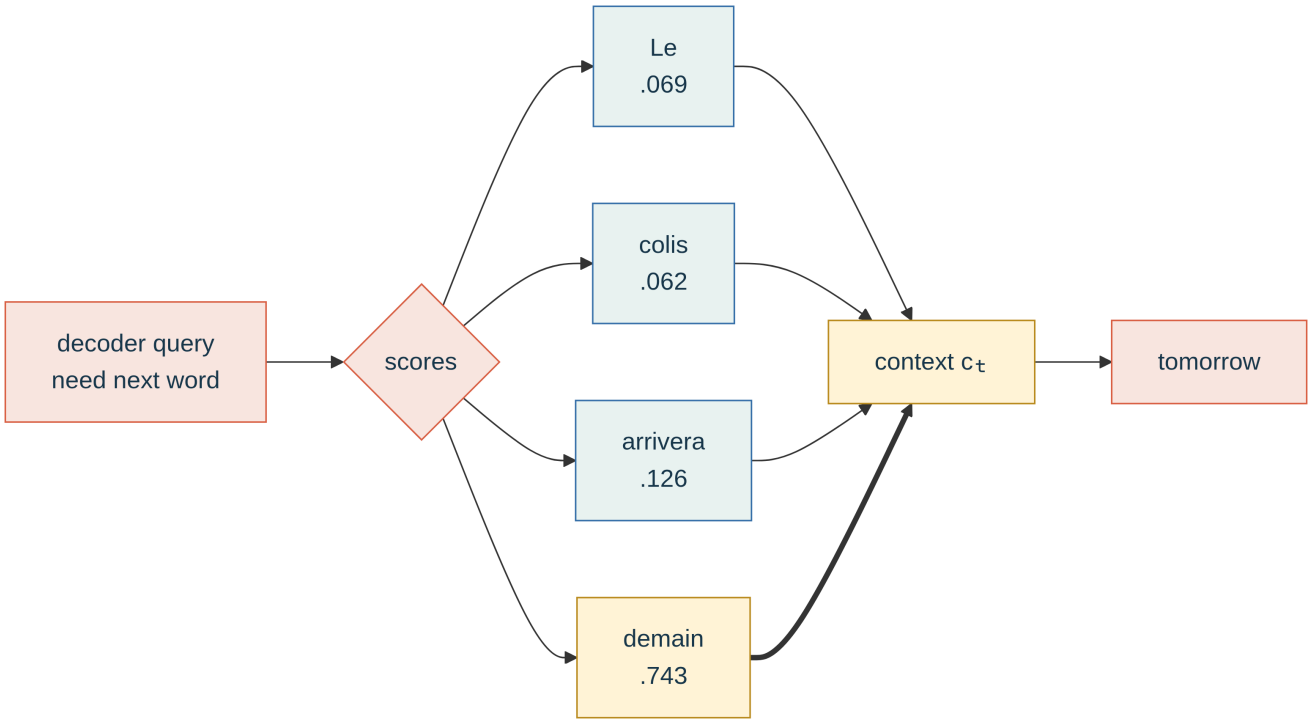


Figure 7: The decoder query for the next target token assigns most weight to the source position containing *demain*.

3.4 A new query creates a new alignment

When the decoder moves from *package* to *will arrive* to *tomorrow*, the source keys and values stay fixed, but the query changes. Therefore the attention distribution changes:

$$\alpha_t = \text{softmax}(q_t K^\top). \tag{13}$$

Table 5: Cross-attention is conditional retrieval, not one global sentence summary.

Target step	Query intent	Likely strongest source key
package	retrieve the translated entity	*colis*
will arrive	retrieve the event	*arrivera*
tomorrow	retrieve the time expression	*demain*

An attention matrix places one decoder query on each row and one source key on each column. Heatmaps of this matrix can reveal plausible alignments, diffuse uncertainty, repeated coverage, or systematic errors.

⚠ An attention heatmap is evidence—not a proof of reasoning

Weights show how values are mixed at a particular attention operation. They do not, by themselves, establish a complete causal explanation of the model prediction. Alternative weight patterns can sometimes produce similar outputs (Jain and Wallace 2019). Use heatmaps as diagnostic evidence together with interventions, errors, and other analyses.

3.5 Section synthesis

Cross-attention links two sequences:

- queries come from the decoder;
- keys and values come from the encoder;
- each target step retrieves a different source context.

The same lookup can operate without a separate encoder and decoder. If queries, keys, and values all come from one sequence, we obtain self-attention.

4 Self-attention

4.1 Change the origin, keep the mechanism

Cross-attention and self-attention use the same score–normalize–retrieve rule. Only the source of the roles changes.

Table 6: Attention is named by where its roles originate.

Mechanism	Queries	Keys	Values
Cross-attention	target/decoder sequence	source/encoder sequence	source/encoder sequence
Self-attention	one sequence	the same sequence	the same sequence

For an input matrix $X \in \mathbb{R}^{n \times d_{model}}$,

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V. \tag{14}$$

The same token vector therefore produces three different learned views: what it asks for, how it advertises itself for matching, and what content it offers if selected.

4.2 A pronoun performs a lookup

Keep the sentence visible:

> The **animal** was tired because **it** ran.

The token *it* is ambiguous in isolation. Its query can compare against every key in the sentence. A plausible distribution is:

Table 7: An illustrative self-attention distribution for contextualizing *it*.

Key token	The	animal	was	tired	because	it	ran
Weight from query *it*	.05	**.55**	.05	.15	.05	.10	.05

The contextual output is

$$z_{it} = .05v_{The} + .55v_{animal} + .05v_{was} + .15v_{tired} + .05v_{because} + .10v_{it} + .05v_{ran}. \tag{15}$$

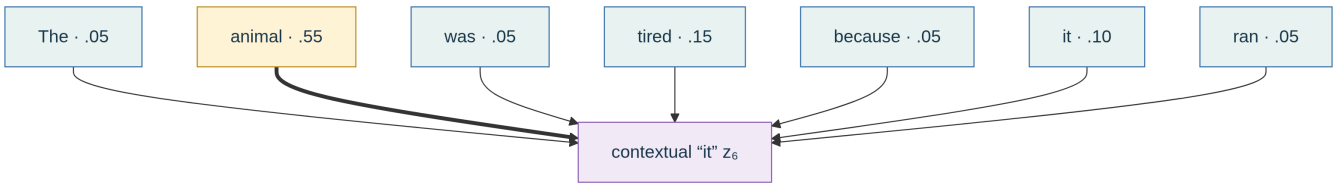


Figure 8: The query at *it* retrieves a weighted mixture from the same sentence, with the strongest illustrative connection to *animal*.

The output at position 6 is no longer a representation of *it* alone. It is *it interpreted in this sentence*.

4.3 Every token asks its own question

Self-attention does not only contextualize the pronoun. Every position i supplies its own query q_i, produces its own distribution over all keys, and receives its own output z_i:

$$z_i = \sum_{j=1}^{n} \alpha_{ij} v_j. \tag{16}$$

All n^2 scores can be computed together:

$$S = QK^\top \in \mathbb{R}^{n \times n}, \tag{17}$$

$$A = \text{softmax}_{row}(S), \qquad Z = AV. \tag{18}$$

For our seven-token sentence, the dimensions are inspectable:

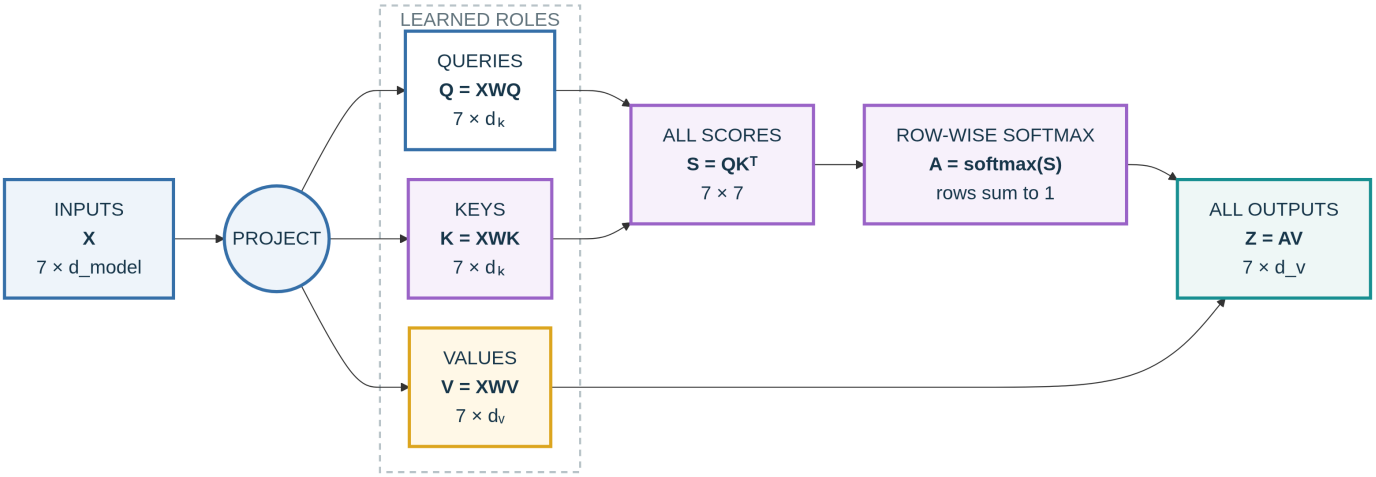


Figure 9: Matrix multiplication performs all seven self-attention lookups together.

Row i of A contains the weights produced by query q_i. Row i of Z is the context retrieved for position i.

4.4 Shorter paths and parallel computation

In a recurrent network, information from position 2 to position 6 crosses four sequential state updates. In one self-attention layer, a learned edge can connect those positions directly.

Table 8: Self-attention changes both the information path and the computational trade-off.

Property	Recurrent layer	Self-attention layer
Long-range path length	grows with distance	one attention edge
Training across positions	sequential	parallel matrix operations
Pairwise interaction cost	linear per recurrent step	quadratic in sequence length
Explicit notion of order	recurrence provides it	must be supplied separately

The quadratic $n \times n$ score matrix is powerful but can become expensive for very long sequences. This limitation motivates later work on sparse, local, linear, and memory-efficient attention. It does not change the basic lookup derived here.

4.5 Section synthesis

Self-attention creates direct, context-dependent token representations and evaluates all queries in parallel. Two practical questions remain:

1. Why do dot products become unstable as vector width grows?
2. How can a language model prevent a position from reading future tokens?

A third modeling question will follow: can one attention distribution capture every useful relation?

5 Modern attention

5.1 Why scale dot products?

Consider a query and key with four matching coordinates:

$$q = [1, 1, 1, 1], \qquad k = [1, 1, 1, 1].$$

Their dot product is

$$q^\top k = 1 + 1 + 1 + 1 = 4.$$

With wider random vectors, more products are summed. Under common independence assumptions, if query and key coordinates have mean zero and variance one, the dot product has variance proportional to d_k:

$$\operatorname{Var}(q^\top k) = d_k.$$

Dividing by $\sqrt{d_k}$ restores an approximately unit variance:

$$\operatorname{Var}\left(\frac{q^\top k}{\sqrt{d_k}}\right) \approx 1.$$

This matters because large logits make softmax extremely sharp. Probabilities approach zero or one, where useful gradients may become small (Vaswani et al. 2017).

! Pause · compare two distributions

Given scores $[4, 2, 0]$ and $d_k = 4$, compare:

$$\operatorname{softmax}([4, 2, 0])$$

with

$$\operatorname{softmax}\left(\frac{[4, 2, 0]}{\sqrt{4}}\right).$$

Which distribution leaves more probability for alternatives?

5.2 Worked scaling example

Since $\sqrt{d_k} = 2$, the scaled scores are $[2, 1, 0]$.

Table 9: Scaling preserves the score ranking but reduces accidental overconfidence.

Candidate	Unscaled score	Unscaled weight	Scaled score	Scaled weight
1	4	**0.867**	2	**0.665**
2	2	0.117	1	0.245
3	0	0.016	0	0.090

The complete operator is therefore

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V. \tag{19}$$

5.3 Causal masking: do not read tomorrow

Suppose an autoregressive language model is trained on:

```
The animal ran home
```

At position 3, the task is to predict the next token *home* from the prefix *The animal ran*. Unrestricted self-attention would allow the query at position 3 to inspect position 4, leaking the answer.

The causal rule is:

$$\text{query position } i \text{ may attend to key position } j \quad \text{only if} \quad j \le i. \tag{20}$$

Define an additive mask M:

$$M_{ij} = \begin{cases} 0, & j \le i, \\ -\infty, & j > i. \end{cases} \tag{21}$$

For four positions:

Table 10: The lower-triangular causal mask.

Query i \ Key j	1	2	3	4
1	□	$-\infty$	$-\infty$	$-\infty$
2	□	□	$-\infty$	$-\infty$
3	□	□	□	$-\infty$
4	□	□	□	□

The masked operation is

$$\widetilde{S} = \frac{QK^\top}{\sqrt{d_k}} + M,$$

$$A = \text{softmax}_{row}(\widetilde{S}), \qquad Z = AV. \tag{22}$$

Because $\exp(-\infty) = 0$, every forbidden future weight becomes zero. All positions can still be processed in parallel during training; the mask controls information, not execution order.

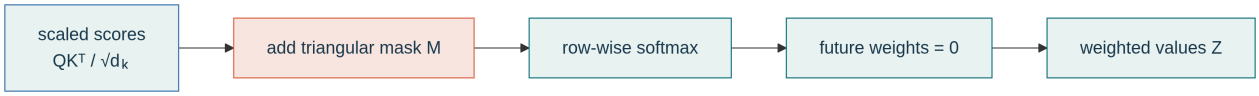


Figure 10: Masking occurs before softmax, converting forbidden future connections into zero attention weight.

5.4 Why use multiple heads?

Consider:

> The **animal** did not **cross** the **street** because **it** was **wide**.

For the token *it*, several relations may be useful:

- reference: *it* → *street*;
- description: *it* → *wide*;
- event participants: *cross* → *animal*, *street*.

A single head produces one attention distribution per query. It can mix several signals, but a multi-head layer provides distinct learned projection spaces:

$$head_r = \text{Attention}(QW_r^Q, KW_r^K, VW_r^V), \tag{23}$$

$$\text{MultiHead}(Q, K, V) = \text{Concat}(head_1, \dots, head_h)W^O. \tag{24}$$

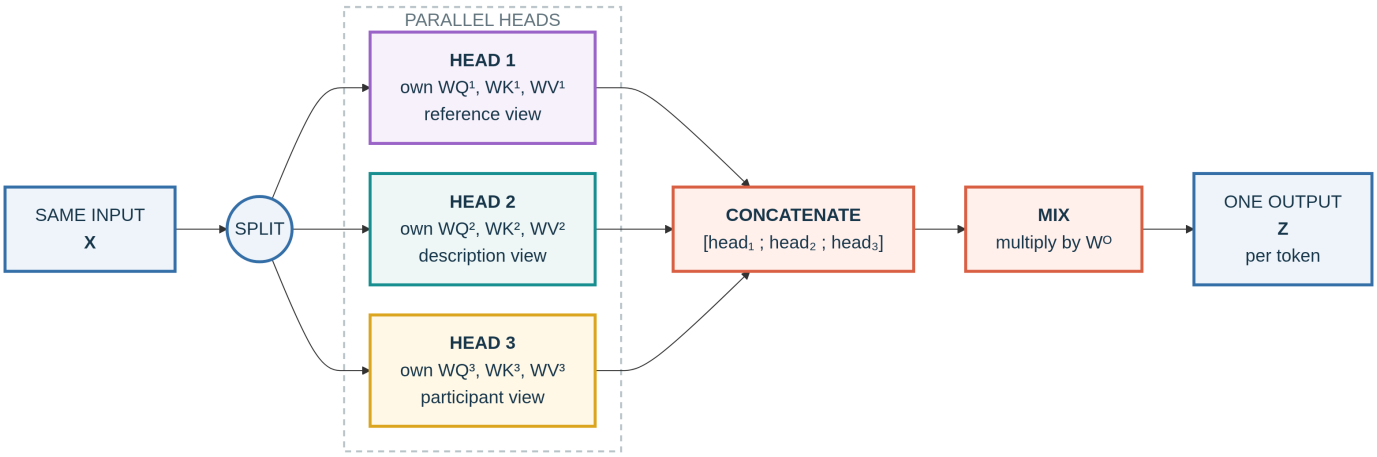


Figure 11: Every head receives the same input but learns separate query, key, and value projections before the results are concatenated and mixed.

The colored linguistic roles are an explanatory possibility, not a constraint. The architecture does not label a head "coreference" or "syntax." Analyses have nevertheless found heads correlated with positional patterns, delimiters, syntactic dependencies, and coreference (Clark et al. 2019).

5.5 Reading attention visualizations

Two common views are useful.

5.5.1 Attention heatmap

Rows represent queries, columns represent keys, and color intensity represents α_{ij}. A heatmap is effective for inspecting all pairwise weights in one head.

5.5.2 Connection view

Tokens are connected by lines whose opacity or width represents attention strength. This view makes a small number of strong relations visually immediate.

BertViz provides head, model, and neuron-oriented views for Transformer attention (Vig 2019). For a research example connecting heads with linguistic relations, read Clark et al., *What Does BERT Look At?* (Clark et al. 2019).

⚠ Interpret with care

An attractive attention line is not automatically the reason for a prediction. Inspect:

- several heads and layers, not one selected picture;
- whether the pattern is stable across examples;
- how changing or removing attended tokens changes the output;
- whether value vectors and downstream layers preserve the apparent relation; and
- whether another attention distribution produces a similar result.

Attention visualization is a valuable microscope, not a complete theory of model reasoning (Jain and Wallace 2019).

5.6 Modern attention in one view

Table 11: Three additions that form the modern attention operator.

Mechanism	Operation	Problem addressed
Scaling	divide logits by $\sqrt{d_k}$	softmax saturation as width grows
Causal mask	add $-\infty$ above the diagonal	future-token leakage
Multiple heads	parallel projected lookups	reliance on one similarity space

The operator is now stable, information-valid, and able to learn several views. It is still not a complete sequence model.

6 The Transformer bridge

6.1 Self-attention has no order by itself

Compare:

```
dog bites man
man bites dog
```

The token set is the same, but the meanings differ. If P is a permutation matrix that reorders positions, self-attention without positional information is permutation equivariant:

$$\mathrm{Attn}(PX) = P\,\mathrm{Attn}(X). \tag{25}$$

Reordering inputs merely reorders outputs. The operation has no independent signal that position 1 came before position 3. A sequence architecture must therefore inject position information. The next chapter will compare ways to do so.

6.2 Attention mixes positions, not features deeply

Attention answers:

> Which positions should exchange information?

After that exchange, each token also needs a nonlinear transformation of its features:

> How should this position transform the information it received?

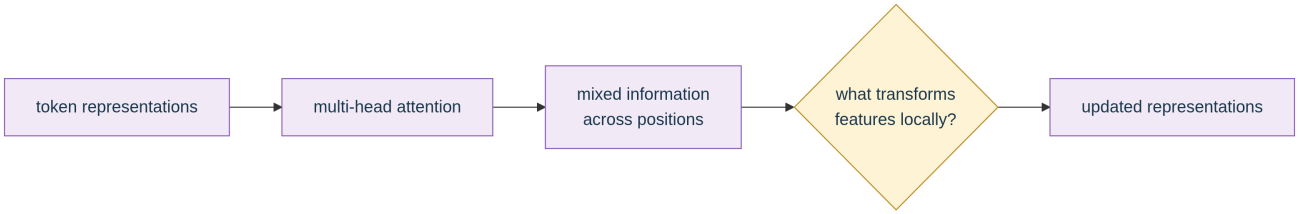


Figure 12: Attention supplies cross-position communication; a complete block also needs position-wise feature transformation.

The Transformer answers this with a position-wise feed-forward sublayer, but its exact equations and placement belong to the architecture chapter.

6.3 Deep stacks need safe routes

One attention layer creates one round of interaction. Rich models stack many layers, which raises two questions:

1. Can an original representation survive repeated transformations?
2. Can activation and gradient scales remain controlled across depth?

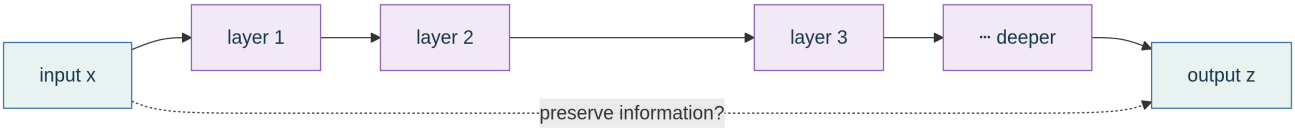


Figure 13: Repeated attention and transformation require a safe route for information and stable activation scales.

Residual pathways and normalization address these requirements in practical Transformer blocks (Vaswani et al. 2017). For now, retain the requirements rather than memorizing a block diagram.

6.4 Final visual bridge

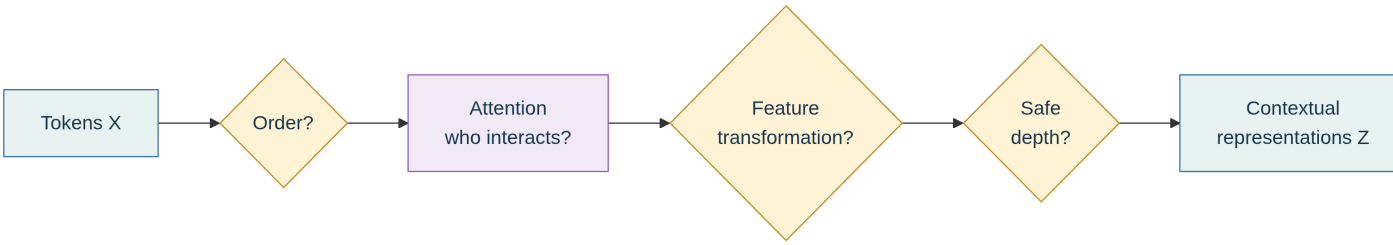


Figure 14: Attention is the core operation, but order, position-wise transformation, and safe depth must be assembled around it.

> **! Question to carry into the next chapter**
>
> How do we assemble order, attention, position-wise transformation, and safe depth into one reusable Transformer block?

7 Chapter summary

The full conceptual chain is now visible:

1. A fixed encoder context creates a storage and access bottleneck.
2. Attention keeps all memories and performs a differentiable soft lookup.
3. Cross-attention lets each decoder query retrieve a different source context.
4. Self-attention lets every token retrieve context from its own sequence.
5. Matrix operations compute all token-to-token scores and weighted sums together.
6. Scaling controls dot-product magnitude.
7. Causal masking prevents future-token leakage.
8. Multiple heads learn parallel projection and retrieval spaces.
9. Attention still needs order, position-wise transformation, and safe depth to become a full architecture.

The most important reusable formula is:

$$\boxed{\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right) V} \tag{26}$$

where $M = 0$ when every connection is permitted and M is a causal or task-specific mask otherwise.

8 Further reading and exploration

8.1 Defining papers

- Bahdanau, Cho, and Bengio · *Neural Machine Translation by Jointly Learning to Align and Translate* introduces learned soft alignment in recurrent encoder–decoder translation (Bahdanau et al. 2015).
- Luong, Pham, and Manning · *Effective Approaches to Attention-based Neural Machine Translation* compares global/local and scoring variants (Luong et al. 2015).
- Vaswani et al. · *Attention Is All You Need* introduces scaled dot-product and multi-head attention within the Transformer (Vaswani et al. 2017).

8.2 Visual explanations

- Jay Alammar · *The Illustrated Transformer* provides a visual walkthrough from attention to the architecture.
- BertViz provides interactive head, model, and neuron views (Vig 2019).
- Tensor2Tensor visualization and implementation preserves an influential historical implementation. The repository is archived and should be treated as legacy code.

8.3 Attention patterns and interpretation

- Clark et al. · *What Does BERT Look At?* studies positional, syntactic, and coreference patterns in BERT heads (Clark et al. 2019).
- Jain and Wallace · *Attention is not Explanation* explains why attention weights alone should not be treated as a faithful explanation (Jain and Wallace 2019).
- Stanford CS224N · Attention and Transformers offers a complementary lecture treatment.

8.4 References

Bahdanau, Dzmitry, Kyunghyun Cho, and Yoshua Bengio. 2015. “Neural Machine Translation by Jointly Learning to Align and Translate.” *International Conference on Learning Representations*. https://arxiv.org/abs/1409.0473.

Clark, Kevin, Urvashi Khandelwal, Omer Levy, and Christopher D. Manning. 2019. “What Does BERT Look at? An Analysis of BERT’s Attention.” *Proceedings of the 2019 ACL Workshop BlackboxNLP*, 276–86. https://doi.org/10.18653/v1/W19-4828.

Jain, Sarthak, and Byron C. Wallace. 2019. “Attention Is Not Explanation.” *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics*, 3543–56. https://doi.org/10.18653/v1/N19-1357.

Luong, Thang, Hieu Pham, and Christopher D. Manning. 2015. “Effective Approaches to Attention-Based Neural Machine Translation.” *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing*, 1412–21. https://doi.org/10.18653/v1/D15-1166.

Sutskever, Ilya, Oriol Vinyals, and Quoc V. Le. 2014. “Sequence to Sequence Learning with Neural Networks.” *Advances in Neural Information Processing Systems* 27. https://papers.nips.cc/paper/5346-sequence-to-sequence-learning-with-neural-networks.

Vaswani, Ashish, Noam Shazeer, Niki Parmar, et al. 2017. “Attention Is All You Need.” *Advances in Neural Information Processing Systems* 30. https://arxiv.org/abs/1706.03762.

Vig, Jesse. 2019. “A Multiscale Visualization of Attention in the Transformer Model.” *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics: System Demonstrations*, 37–42. https://doi.org/10.18653/v1/P19-3007.